



Fully Connected Feedforward Neural Networks

Dr. Sunil Kumar Vuppala

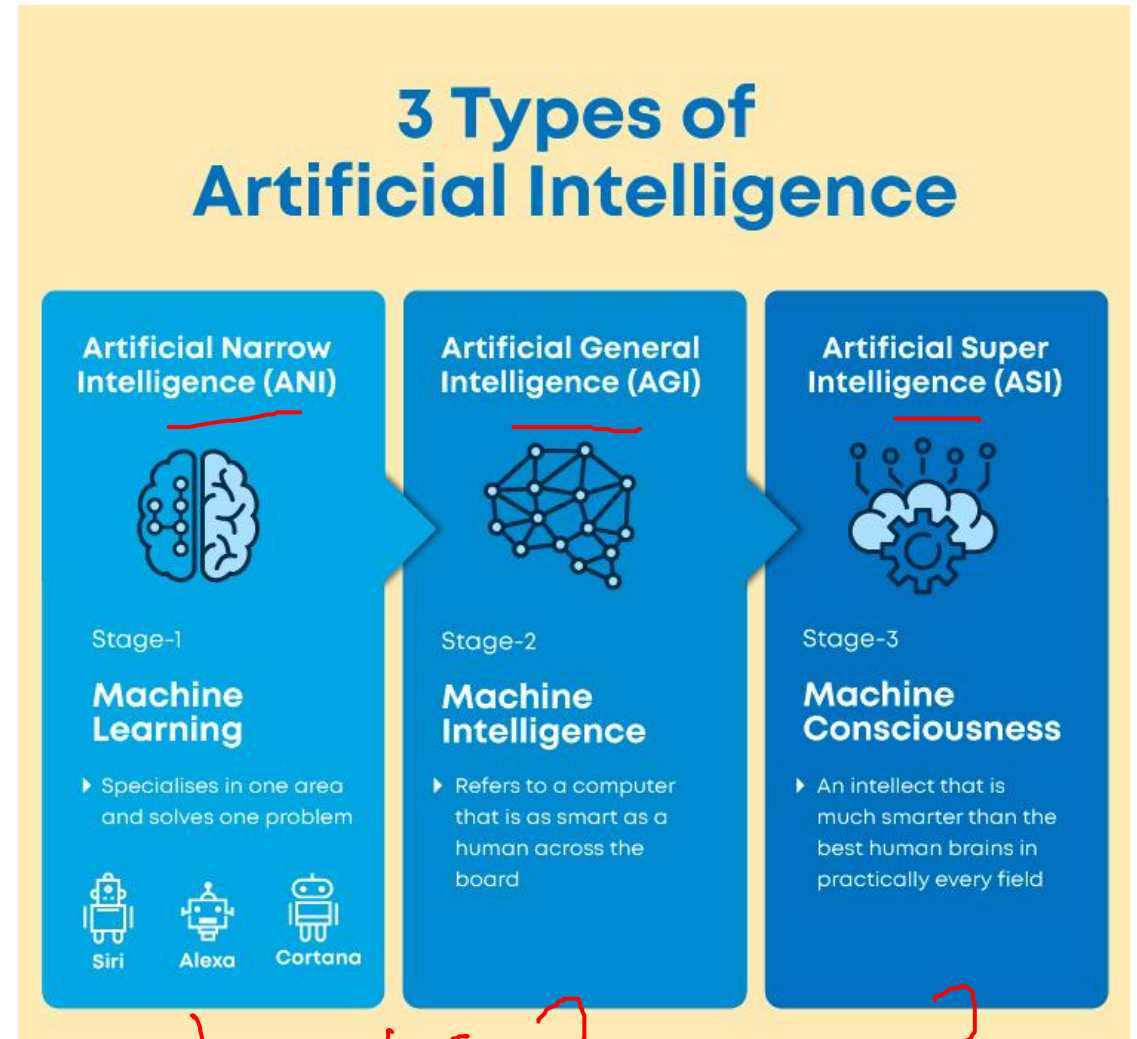
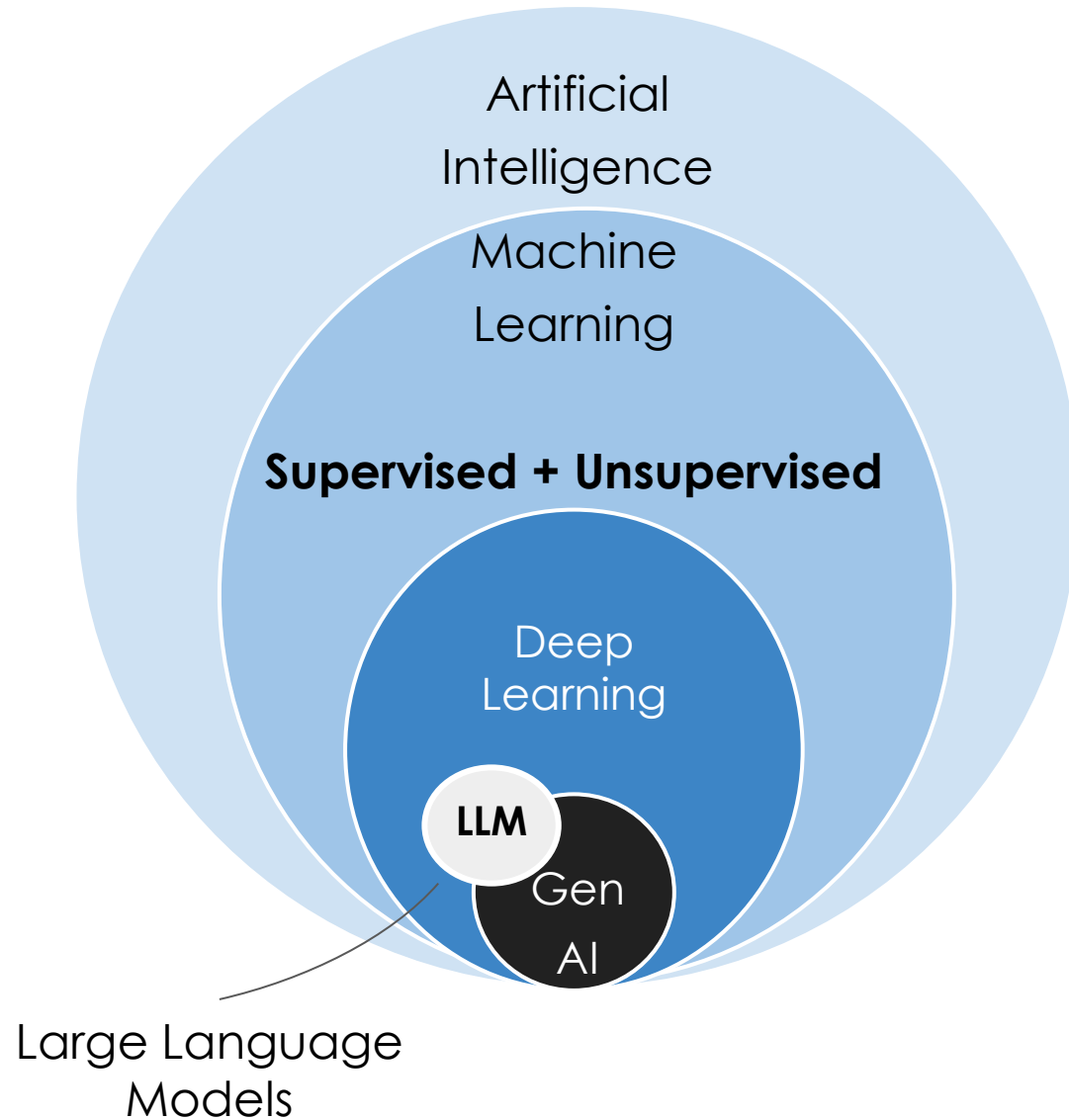
AI Expert, ACM Distinguished Speaker

www.linkedin.com/in/sunilvuppala/



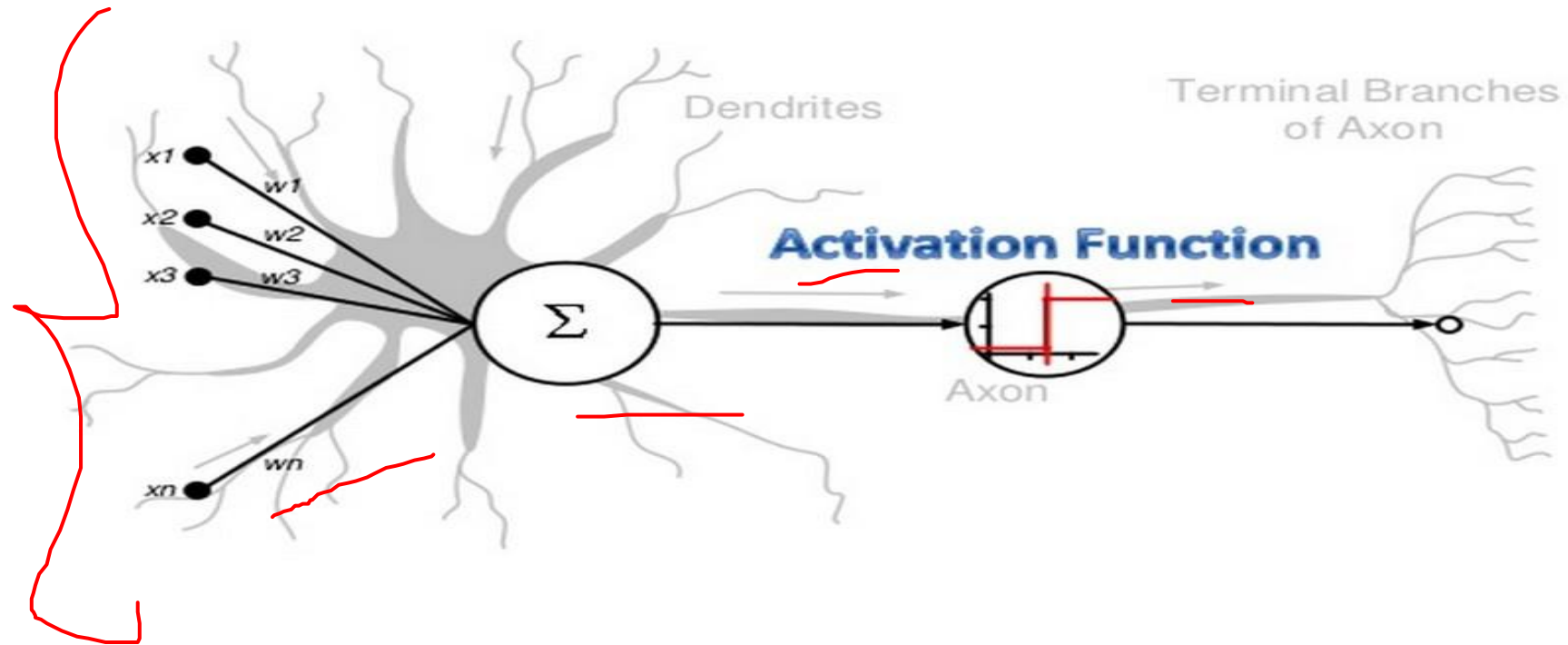
Contents

- Recap of Deep Learning
- Structure of a Neural Network
- How Deep Learning works?
 - Feedforward in NN
 - Backpropagation in Neural Networks – Network simulator
- Overfitting
- Hyperparameter Finetuning



1 1.5 2 3

Artificial neural network



Slide credit : Andrew L. Nelson

Artificial neural network

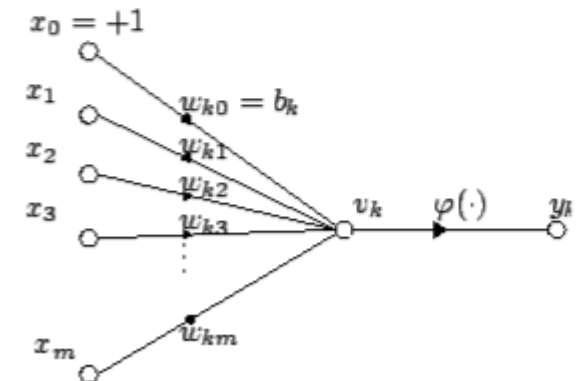
Artificial **neurons** are elementary units in an artificial neural network. The artificial neuron receives one or more **inputs** (representing dendrites) and sums them to produce an **output** (or activation) (representing a neuron's axon).

Usually the sums of each node are weighted, and the sum is passed through a non-linear function known as an **activation** function.

The output of the k th neuron is:

$$y_k = \varphi \left(\sum_{j=0}^m w_{kj} x_j \right)$$

Where φ (phi) is the transfer function.



Learning process

These equations

$$3x + 2y + 5z = 7$$

$$1x + 3y + 9z = 2$$

$$4x + y + 0z = 8$$

can be written as a dot product of a matrix and a vector:

$$\begin{bmatrix} 3 & 2 & 5 \\ 1 & 3 & 9 \\ 4 & \frac{1}{2} & 0 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} 7 \\ 2 \\ 8 \end{bmatrix}$$

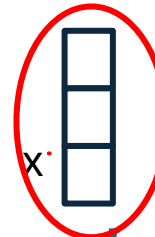
x

$$Ax = B$$

Parameters
(things we're
learning)

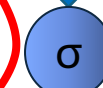


$\cdot$

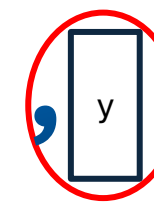


Input feature
vector

Update
parameter
s



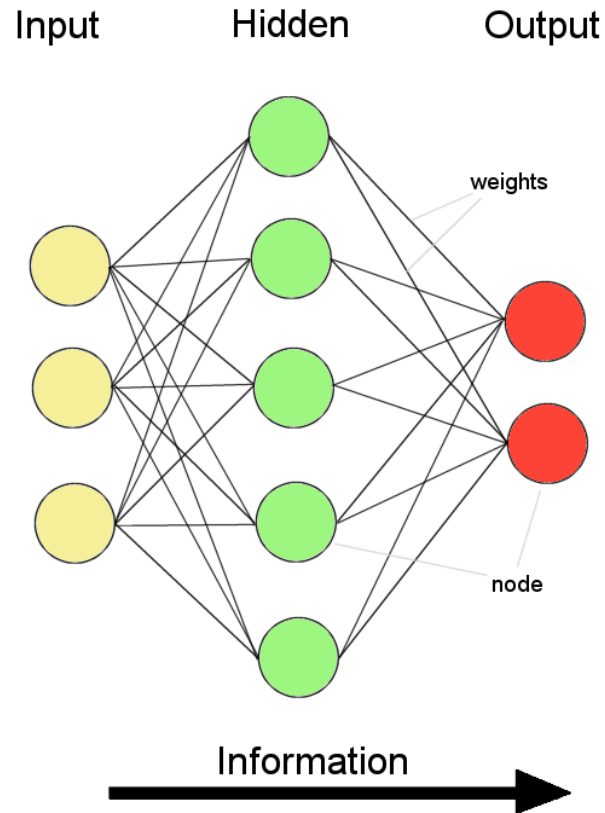
cost



Actual
value

$$\frac{600 - 50}{60}$$

Feed forward nets



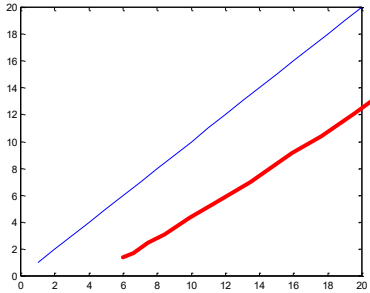
Information flow is unidirectional

- Data is presented to *Input layer*
 - Passed on to *Hidden Layer*
 - Passed on to *Output layer*
- Information is distributed
 - Information processing is parallel
 - Backpropagation
- Requires training set (input / output pairs)
 - Starts with small random weights
 - Error is used to adjust weights (supervised learning)

What are hidden layers?

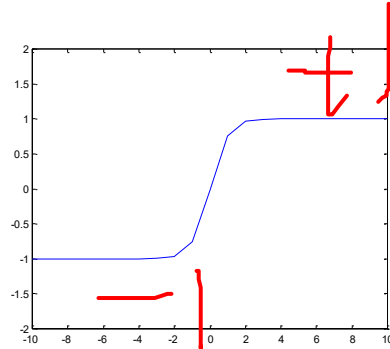
- They are non-linear sums of the inputs
- So, they are not linearly dependent on inputs (they are non-linearly dependent)
- They are engineered features of the original inputs!
 - Feature engineering is so difficult because for each type of data and each type of problem, different features do well
 - Neural networks can potentially **build features hierarchically**

Activation functions



Linear

$$y = x$$



Rectifier / ramp function

$$f(x) = \max(0, x)$$

x is the input to a neuron.

smooth approximation to the rectifier

is softplus

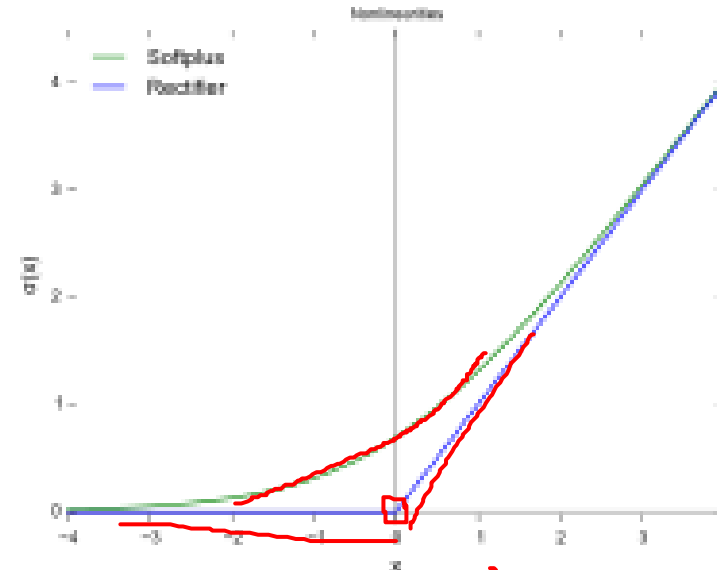
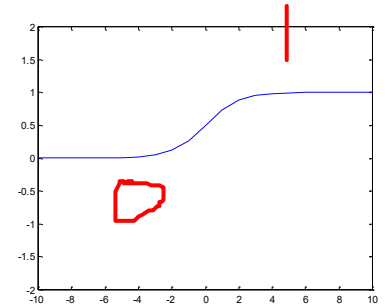
$$f(x) = \ln(1 + e^x)$$

Hyperbolic tangent

$$y = \frac{\exp(x) - \exp(-x)}{\exp(x) + \exp(-x)}$$

Logistic

$$y = \frac{1}{1 + \exp(-x)}$$



$$(0, x)$$

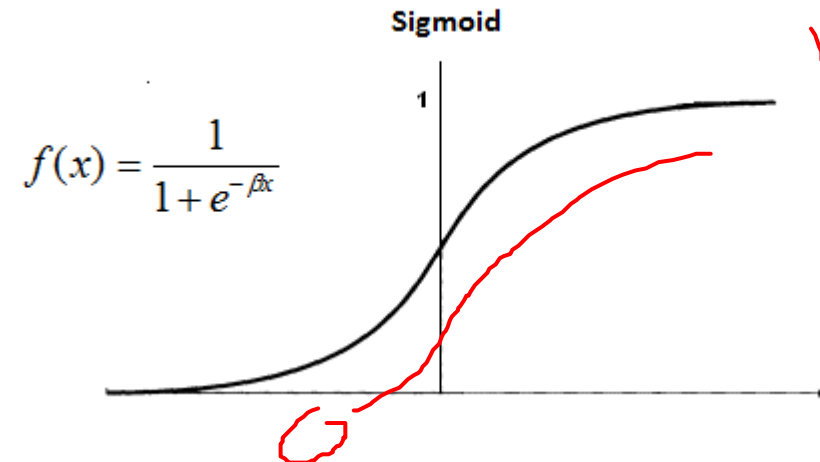
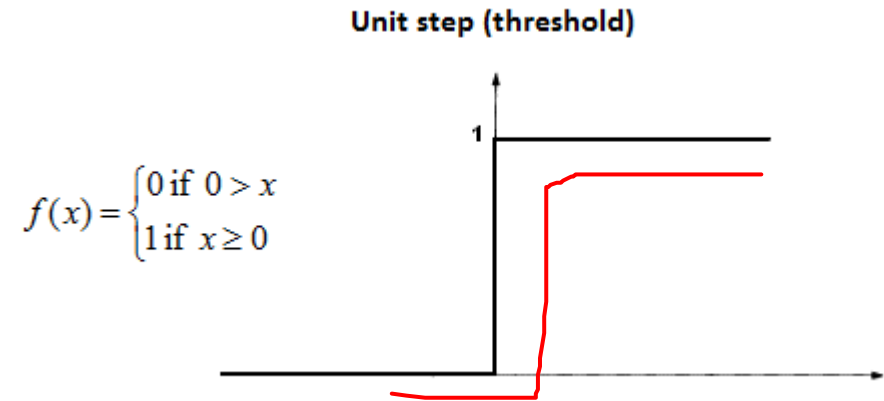
Activation functions

Unit step (threshold):

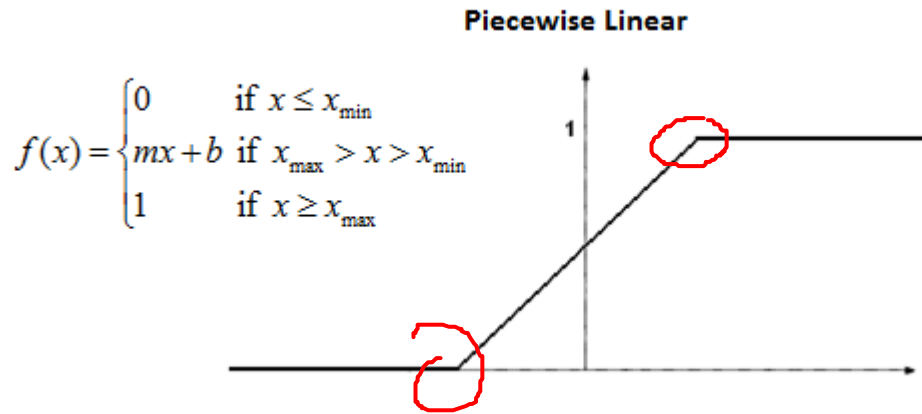
The transfer function translates the input signals to output signals. Four types of transfer functions are commonly used, Unit step (threshold), sigmoid, piecewise linear, and Gaussian.

Sigmoid:

The sigmoid function consists of 2 functions, logistic and tangential. The values of logistic function range from 0 and 1 and -1 to +1 for tangential function.

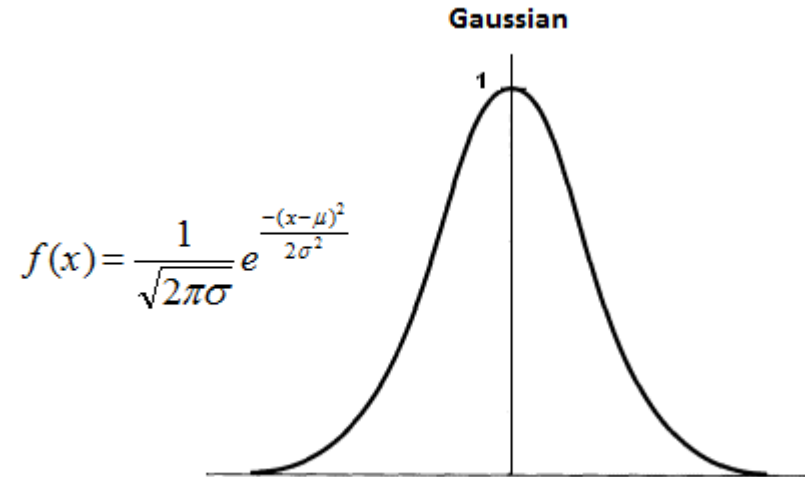


Activation functions



Piecewise Linear

The output is proportional to the total weighted output.



Gaussian

Gaussian functions are bell-shaped curves that are continuous. The node output (high/low) is interpreted in terms of class membership (1/0), depending on how close the net input is to a chosen value of average.

Properties of activation function

$$x^2 = 2x$$

$$\frac{\partial}{\partial x} x + y$$

$$x=1 \quad f(0)=1$$

$$2$$

- **Nonlinear:**

- When the activation function is non-linear, then a two-layer neural network can be proven to be a universal function approximator.

- **Continuously differentiable:**

- This property is necessary for enabling gradient-based optimization methods.
- The binary step activation function is not differentiable at 0, and it differentiates to 0 for all other values, so gradient-based methods can make no progress with it.

- **Range:**

- When the range of the activation function is finite, gradient-based training methods tend to be more stable, because pattern presentations significantly affect only limited weights.
- When the range is infinite, training is generally more efficient because pattern presentations significantly affect most of the weights. Smaller learning rates are typically necessary.

- https://en.wikipedia.org/wiki/Activation_function

Properties of activation function

- **Monotonic:**

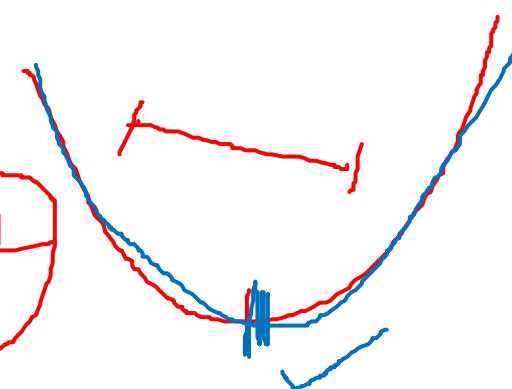
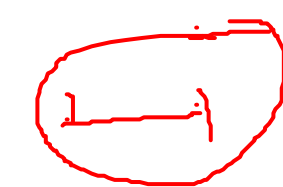
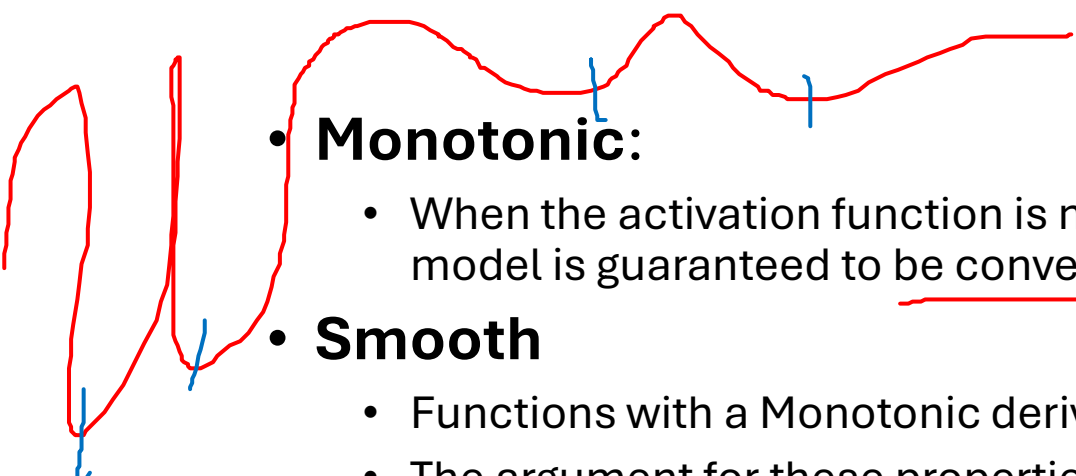
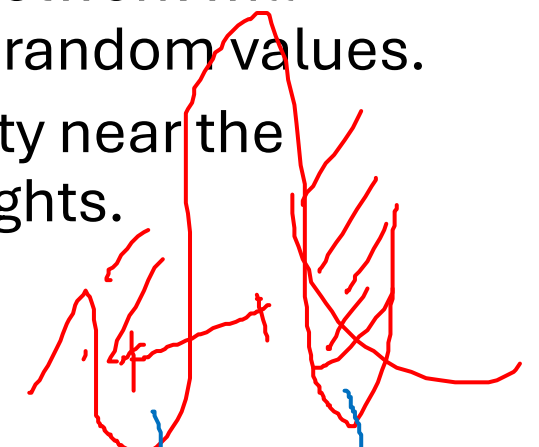
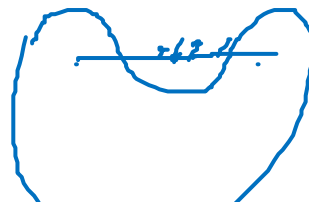
- When the activation function is monotonic, the error surface associated with a single-layer model is guaranteed to be convex.

- **Smooth**

- Functions with a Monotonic derivative have been shown to generalize better in some cases.
- The argument for these properties suggests that such activation functions are more consistent with Occam's razor.

- **Approximates identity near the origin:**

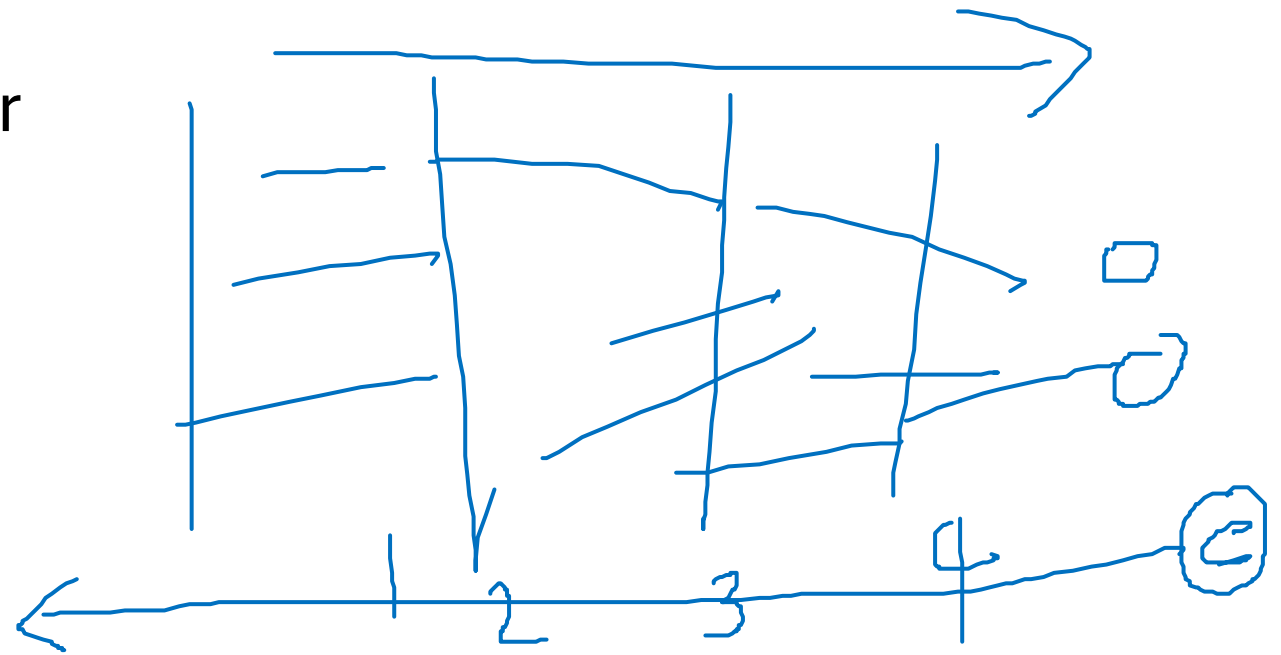
- When activation functions have this property, the neural network will learn efficiently when its weights are initialized with small random values.
- When the activation function does not approximate identity near the origin, special care must be used when initializing the weights.



But, how to learn hidden weights

- We need $\frac{\partial E}{\partial w}$. But, what is the error in internal nodes?

- Back propagation is the answer



Back propagation

3

- With the concept of **gradient descent**

- Forward propagation: Activation [input * weight matrix]
- To get optimal value of each weight:
 - Direction is opp. to the gradient
 - Find weight with minimum error
 - Derivative (slope of a tangent line - rate of change of a function)
 - Partial derivative (wrt one of the variables)
 - Chain rule (derivatives of composite functions)
 - calculate the error wrt each weight

1
2
4

- **New weight = old weight - Derivative Rate * learning rate**

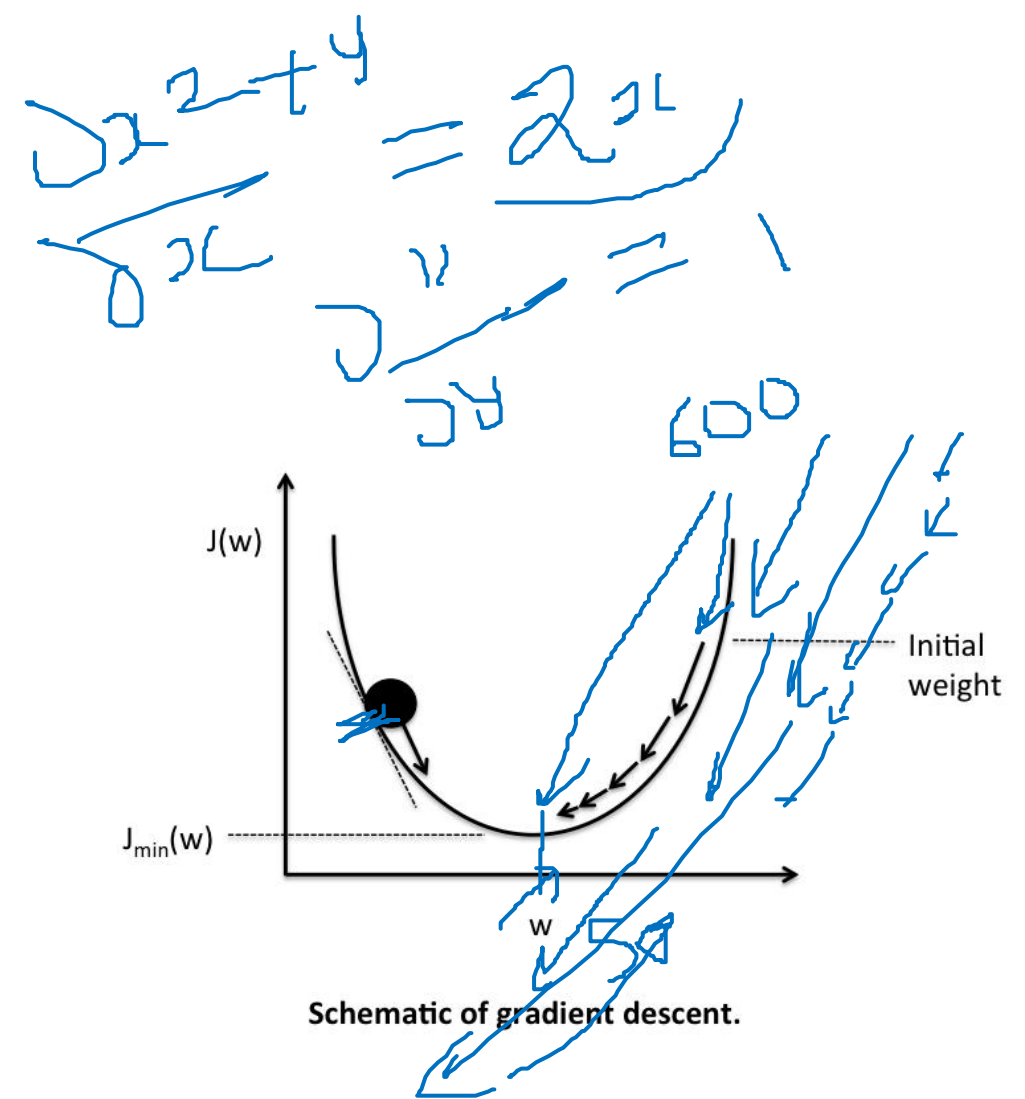
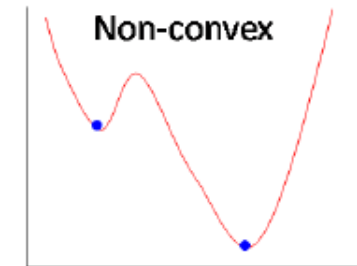
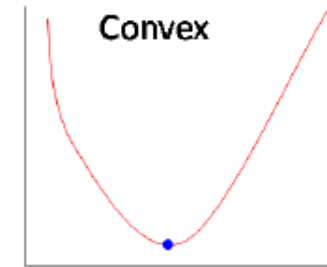


Fig Ref:-

<http://datathings.com/blog/post/neuralnet/>

Gradient descent

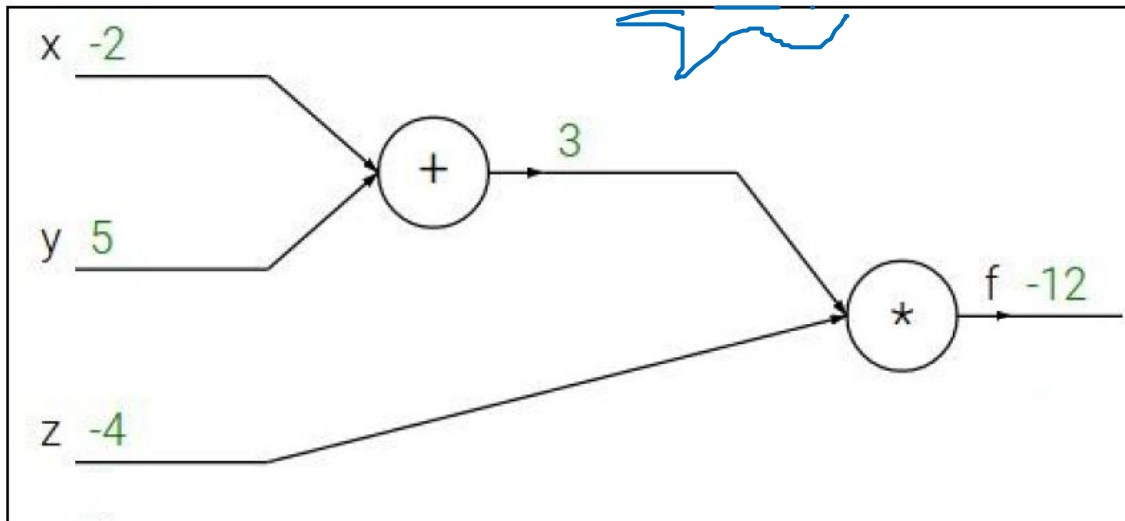
- Randomly start somewhere and compute slope using calculus. Update the coefficients in the negative slope direction
- Repeat until convergence
- Gradient descent is greedy
- Multiple initiations and selection of the best is the option
- No way to theoretically guarantee convergence to global optimum.
- But, generally convergence to a local optimum which is very close to global.
- Therefore state-of-the-art results :)



Back propagation algorithm

There were no training algorithms for multi layered networks until 80s. Geoff Hinton, then a young professor at Carnegie-Mellon University, solved it with back-propagation algorithms. He used chain rule of differentiation to propagate the error backwards and update the weights.

$$f(x, y, z) = (x + y)z$$

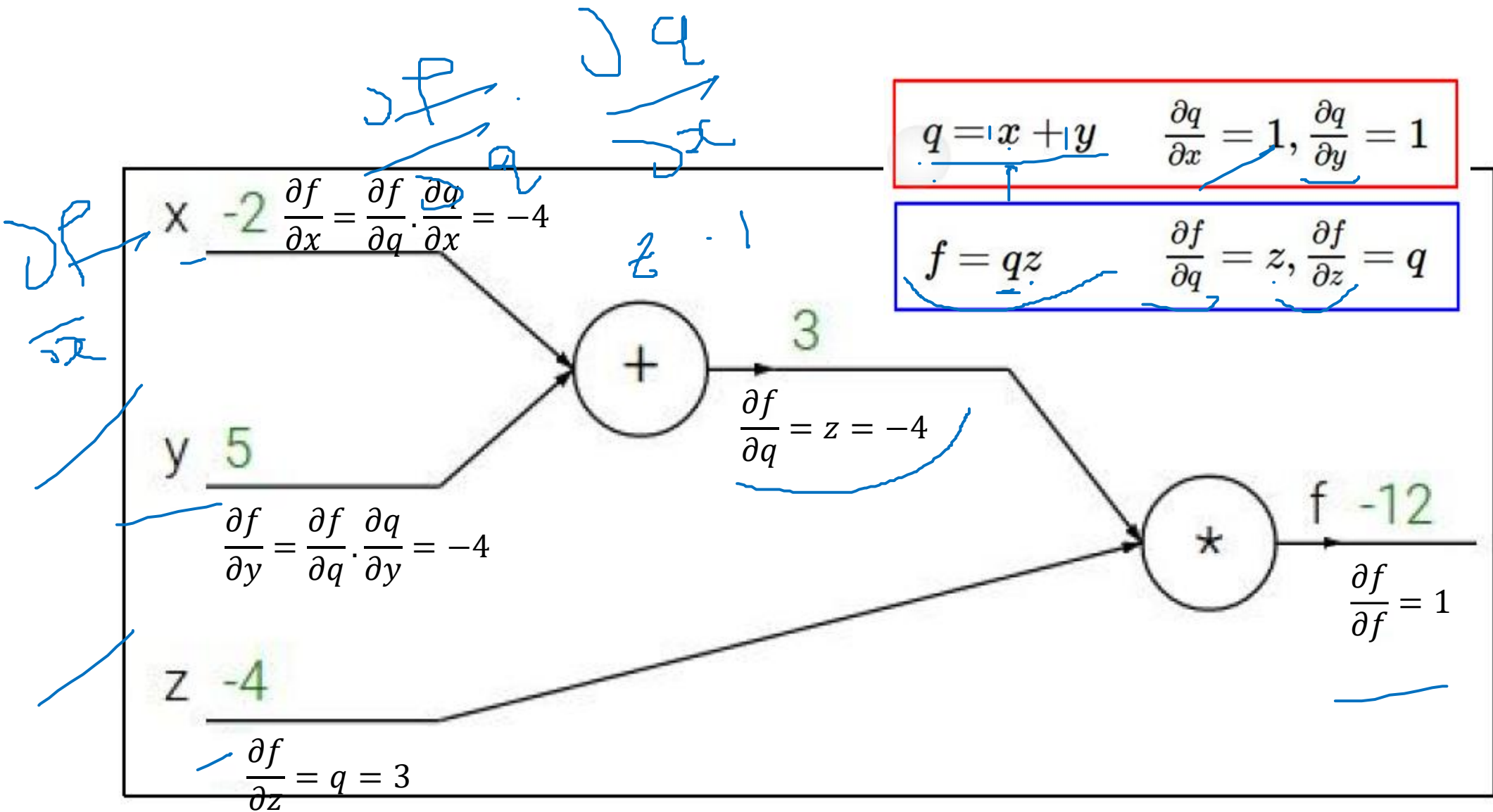


$$f = (x + y)z$$

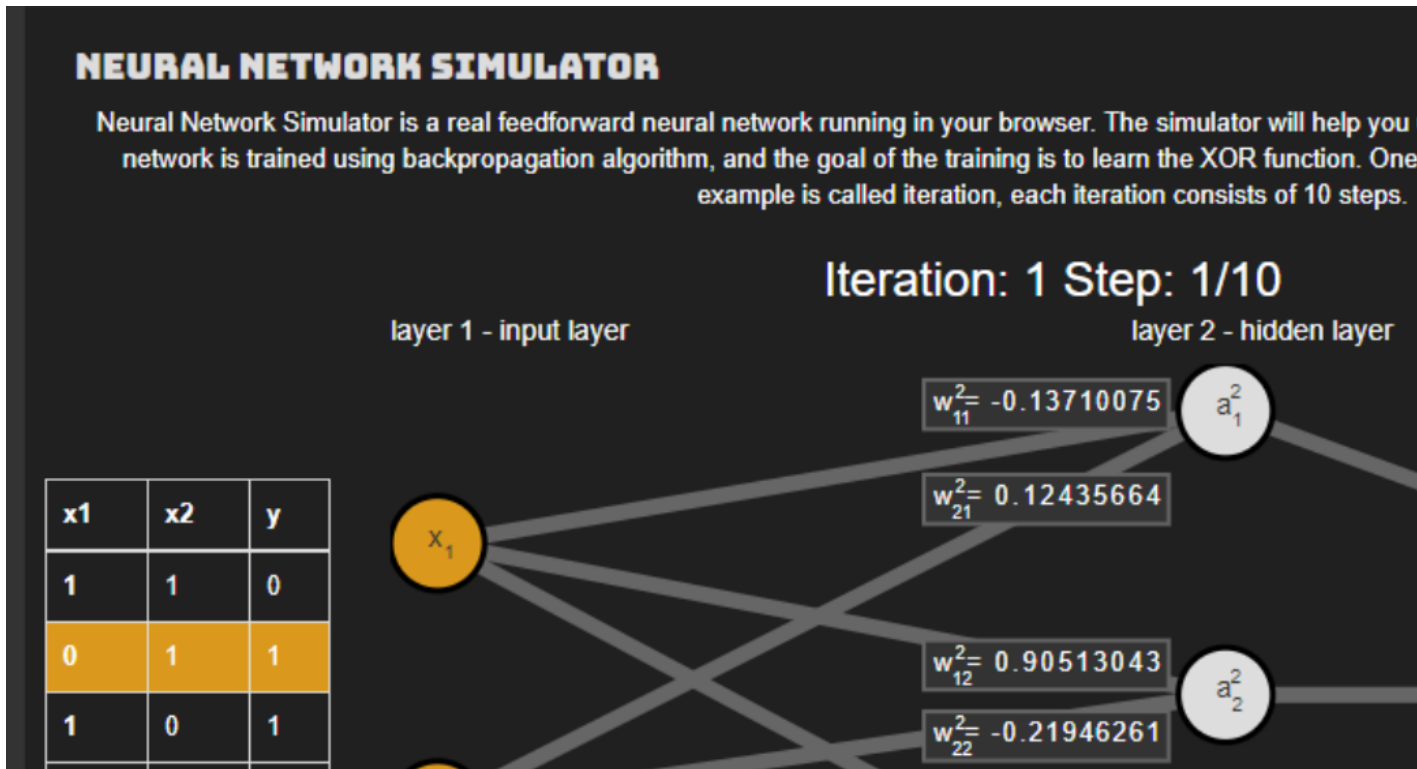
$$x = -2, y = 5, z = -4$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$

$$\frac{\partial f}{\partial x} = \frac{\partial}{\partial x} (x + y)z = z$$

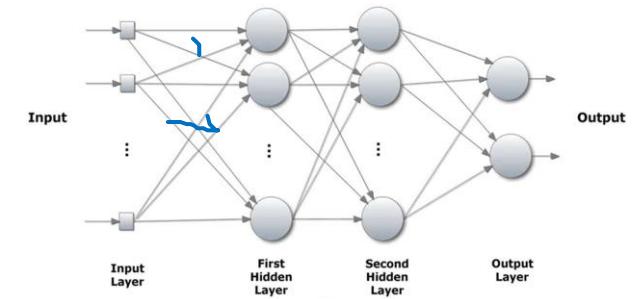


Neural Network simulator



- www.mladdict.com/neural-network-simulator
- <https://mattmazur.com/2015/03/17/a-step-by-step-backpropagation-example/>

Artificial neural networks



Output of one layer is input to another.

Layer1: $\underline{h_1 = f_1(x) = \sigma(w_1 x + b_1)}$

Layer2: $h_2 = f_2(h_1) = \sigma(w_2 h_1 + b_2)$

Layer3: $h_3 = f_3(h_2) = \sigma(w_3 h_2 + b_3)$

.

.

.

LayerN: $\text{output} = f_N(h_{N-1}) = \sigma(w_N h_{N-1} + b_N)$

$\text{output} = f_N(f_{N-1}(f_{N-2}(\dots(f_1(x)))) = f(x)$

0.2

Avoid Over Fitting

- Dropouts
- Early stopping
- Use Data Augmentation
- Use regularization

L1
L2

SMOTE →

GANs

Generative →

Gen AB models

Thank you

Happy Learning

[Linkedin.com/in/sunilvuppala](https://www.linkedin.com/in/sunilvuppala)